

Marlena Kuhn

Machine Learning in Medicine and Biology

Homework 4 - Deep Learning for Fun and Profit

Part A - Backpropagation

A.3

$$\begin{array}{c} \mathbf{x} \\ \begin{bmatrix} 1 \\ 2 \end{bmatrix} \end{array} \quad \mathbf{w} \begin{bmatrix} .1 & -.2 & 0 \end{bmatrix} \quad \mathbf{z} \begin{bmatrix} -.3 \end{bmatrix} \approx \mathbf{a} \begin{bmatrix} -.3 \end{bmatrix} \quad \mathbf{y} \begin{bmatrix} 1 \end{bmatrix}$$

$\hat{y}$

1) $z = w_1 * x_1 + w_2 * x_2 + b$

$$= .1(1) + (-.2)(2) + 0 = .10 - .40 = -.30$$

$$\boxed{z = -0.3}$$

→ generally, a or $\hat{y}$ would be z after ReLU (which would be 0)

However,

$$\hat{y} = z = -.30$$

$$\boxed{\hat{y} = -0.3}$$

2) $L = \frac{1}{2}(\hat{y} - y)^2$

$$= \frac{1}{2}(-.30 - 1.0)^2 = \frac{1}{2}(-1.3)^2 = \frac{1}{2}(1.69) = .845$$

$$\boxed{L = 0.845}$$

(cont.) →

3)

$$\begin{array}{c}
 \begin{array}{|c|} \hline x \\ \hline 1 \\ \hline 2 \\ \hline \end{array}
 \end{array}
 \begin{array}{c}
 \longrightarrow \\
 \longrightarrow
 \end{array}
 \begin{array}{|c|} \hline \frac{\partial L}{\partial w} \\ \hline -1.3 \\ \hline -2.6 \\ \hline -1.3 \\ \hline \end{array}
 \begin{array}{c}
 \\
 \frac{\partial L}{\partial b}
 \end{array}$$

$$w \quad \begin{array}{|c|c|c|} \hline .1 & -.2 & 0 \\ \hline \end{array}
 \quad \begin{array}{|c|} \hline z \\ \hline -.3 \\ \hline \end{array}
 =
 \begin{array}{|c|} \hline -.3 \\ \hline \end{array}
 -
 \begin{array}{|c|} \hline 1 \\ \hline \end{array}
 =
 \begin{array}{|c|} \hline -1.3 \\ \hline \end{array}$$

$\hat{y}$ y

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = -.3 - 1.0 = -1.3$$

$$z = \hat{y}, \quad \frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} = -1.3$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} \times 1 = -1.3 \times 1 = -1.3$$

~~max~~

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \times X$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z} \times x_1 = -1.3 \times 1 = -1.3$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z} \times x_2 = -1.3 \times 2 = -2.6$$

ϵ_1

ϵ_2

$$\boxed{\frac{\partial L}{\partial \hat{y}} = -1.3, \quad \frac{\partial L}{\partial z} = -1.3, \quad \frac{\partial L}{\partial b} = -1.3, \quad \frac{\partial L}{\partial w_1} = -1.3, \quad \frac{\partial L}{\partial w_2} = -2.6}$$

$$4) \quad w_{1, \text{new}} = w_1 - \eta \left(\frac{\partial L}{\partial w_1} \right) = .1 - .1(-1.3) = .1 + .13 = .23$$

$$w_{2, \text{new}} = w_2 - \eta \left(\frac{\partial L}{\partial w_2} \right) = -.2 - .1(-2.6) = -.2 + .26 = 0.06$$

$$b_{\text{new}} = b - \eta \left(\frac{\partial L}{\partial b} \right) = 0 - .1(-1.3) = 0 + .13 = 0.13$$

$$\boxed{w_{1, \text{new}} = 0.23, \quad w_{2, \text{new}} = 0.06, \quad b_{\text{new}} = 0.13}$$

Part B - Colab Notebook

1) Step 1 is to clear the state of the (recorded) gradients between the batches that the data was split in to. This starts the calculations of each batch as their own, rather than a continuation of the previous batch's calculations.

Step 2 is getting the predicted y (or output) from the model.

Step 3 calculates the loss, or the difference between the predicted y and the actual y .

Step 4 performs back propagation on the model. This uses the difference between predicted y and actual y (from Step 3) to get $\frac{\partial L}{\partial z}$, $(\frac{\partial L}{\partial a},)$ $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial w}$ for each level of the model.

Step 5 gets the new weights (and biases) based on Step 4's calculations. This can be represented by $w_{\text{new}} = w - \text{learning rate}(\frac{\partial L}{\partial w})$ (and $b_{\text{new}} = b - \text{learning rate}(\frac{\partial L}{\partial b})$).

These steps can induce "learning" by causing the model to update and become iteratively better/more accurate. These five steps basically complete the backpropagation by hand steps in part A (reset gradients, forward pass, loss, back-propagation, and gradient descent updates) and allow the model to "learn" and change from its mistakes.

2) Residual connection in a neural net is the re-introduction of the original input or output of a (more) distant hidden layer before the neural net finalizes the output. This allows the output to be more-closely related/influenced by (more) unmodified data. You might want a residual connection between layers because it could produce a more accurate and grounded final output.

3) The label had to be converted from a string to an integer to a tensor because it allowed the label to be moved to the GPU.

4) The observation type they both address CO (carbon monoxide) level in terms of low (< 2.0), medium (2.0 and < 5.0) and high (> 5.0). The 1D CNN and RNN take in the same data and both predict ~~CO(GT)~~ amount class for a subsequent hour based on the previous 24-hour window. One model is better than the other when its validation accuracy (correct/total) is higher. The 1D CNN is better when ^{number of} epochs is 2, 5, 6, 7, 8, and 10. RNN is better when ^{number of} epochs is 1, 3, ~~4~~, and 9. (This could be loosely) generalized to RNN is better when there are 4 or fewer epochs, and 1D CNN is better when there are 5 or more epochs.)